การทดลอง

เรื่อง Sequential vs. Parallel Performance

จัดทำโดย

นายวรดร พรมอนันต์ รหัส 67543206020-9

ห้อง ENGCE200_SEC 1

เสนอ

อาจารย์นุรักษ์ ไชยศรี

ใบงานนี้เป็นส่วนหนึ่งของวิชา

ENGCE125

ระบบปฏิบัติการ

(Operating Systems)

หลักสูตร วศ.บ. วิศวกรรมคอมพิวเตอร์

สาขาวิชาวิศวกรรมไฟฟ้า คณะ วิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา ภาคพายับ เชียงใหม่

ภาคเรียนที่ 2 ปีการศึกษา 2568

/* Main Class */

```
import java.util.Arrays;

public class MainFunction {

    public static void main(String[] args) {

        //create Data

        int size = 1000;

        int lengthOfDataRandom = 1_000_00;

        int[] numbers1 = new int[size]; //reference type

        int[] numbers2 = new int[size]; //reference type

        for (int i = 0; i < size; i++) {

            numbers1[i] = (int) (Math.random() * lengthOfDataRandom);

            numbers2[i] = numbers1[i] ;

        }//end for

        viewCore V = new viewCore();

        Sequential seq = new Sequential();

        long startTime;

        long endTime;

        long duration;

        //Sequential

        System.out.println("Normal Array: " + Arrays.toString(numbers1));

        startTime = System.nanoTime(); //START

        seq.sortSequentialAndStats(numbers1);

        endTime = System.nanoTime(); //STOP

        duration = endTime - startTime;

        System.out.println("Sorted Array: " + Arrays.toString(numbers1));

        System.out.println("Meaning time (milliseconds): " + (duration /
1_000_000.0) + " ms");
```

```
        System.out.println("//---------------------------------//");

        //Data parallelism

        System.out.println("Original Array: " + Arrays.toString(numbers2));

        startTime = System.nanoTime(); //START

        V.dataParallelism(numbers2);

        endTime = System.nanoTime(); //STOP

        duration = endTime - startTime;

        System.out.println("Sorted Array: " + Arrays.toString(numbers2));

        System.out.println("Meaning time (milliseconds): " + (duration /
1_000_000.0) + " ms");

        //Task parallelism

        System.out.println("//----------------------------------//");

        startTime = System.nanoTime(); //START

        V.taskParallelism(numbers2);

        endTime = System.nanoTime(); //STOP

        duration = endTime - startTime;

        System.out.println("Meaning time (milliseconds): " + (duration /
1_000_000.0) + " ms");

    }

}
```

/* Parallelism */

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicLong;
public class viewCore {
    public void core() {
        int size = 100;
        int cores = Runtime.getRuntime().availableProcessors();
        System.out.println(cores + " Cores");
        int chunkSize = size / cores;
        System.out.println(chunkSize);
    }
    public void dataParallelism(int[] data) {
        int size = data.length;
        QuickSort sort = new QuickSort();
        int cores = Runtime.getRuntime().availableProcessors();
        ExecutorService executor = Executors.newFixedThreadPool(cores);
        int chunkSize = size / cores;
        for (int i = 0; i < cores; i++) {
            int start = i * chunkSize;
            //if last = size else = (i + 1) * chunkSize
            int end = (i == cores - 1) ? size : (i + 1) * chunkSize;
            executor.submit(() -> {
                sort.quickSort(data, start, end - 1);
            });//end function
        }//end for
        executor.shutdown();
```

```
        try {
            if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
                executor.shutdownNow();
            }
        } catch (InterruptedException e) {
            executor.shutdownNow();
        }
        // merge data
        int currentSortedEnd = chunkSize - 1;
        for (int i = 1 ; i < cores ; i++) { //1-11
            // Find the end point of the next chunk to merge
            int nextChunkEnd = (i == cores - 1) ? size - 1 : (i + 1) *
chunkSize - 1;
            sort.merge(data, 0, currentSortedEnd, nextChunkEnd);
            currentSortedEnd = nextChunkEnd;
        }
    }
    public void taskParallelism(int[] data) {
        AtomicLong max = new AtomicLong(Long.MAX_VALUE);
        AtomicLong min = new AtomicLong(Long.MIN_VALUE);
        AtomicLong avg = new AtomicLong(0);
        AtomicLong mean = new AtomicLong(0);
        double[] med = new double[1];

        int cores = Runtime.getRuntime().availableProcessors();
        ExecutorService executor = Executors.newFixedThreadPool(cores);
```

```
//MAX
executor.submit(() -> {
    max.set(taskMax(data));
});
//Min
executor.submit(() -> {
    min.set(taskMin(data));
});
//AVG
executor.submit(() -> {
    avg.set(taskAvg(data));
});
//Mean
executor.submit(() -> {
    mean.set(taskAvg(data));
});
executor.submit(() -> {
    med[0] = median(data);
});
executor.shutdown();
try {
    if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
        executor.shutdownNow();
    }
} catch (InterruptedException e) {
    executor.shutdownNow();
}
```

```
    System.out.println("max = " + max);
    System.out.println("Min = " + min);
    System.out.println("AVG = " + avg);
    System.out.println("Mean = " + mean);
}//end taskParallelism
public long taskMax(int[] data) {
    long max = 0;
    for (int i = 0; i < data.length; i++) {
        if (data[i] > max) {
            max = data[i];
        }
    }//end for
    return max;
}
public long taskMin(int[] data) {
    long min = 9_999_999;
    for (int i = 0; i < data.length; i++) {
        if (data[i] < min) {
            min = data[i];
        }
    }//end for
    return min;
}
```

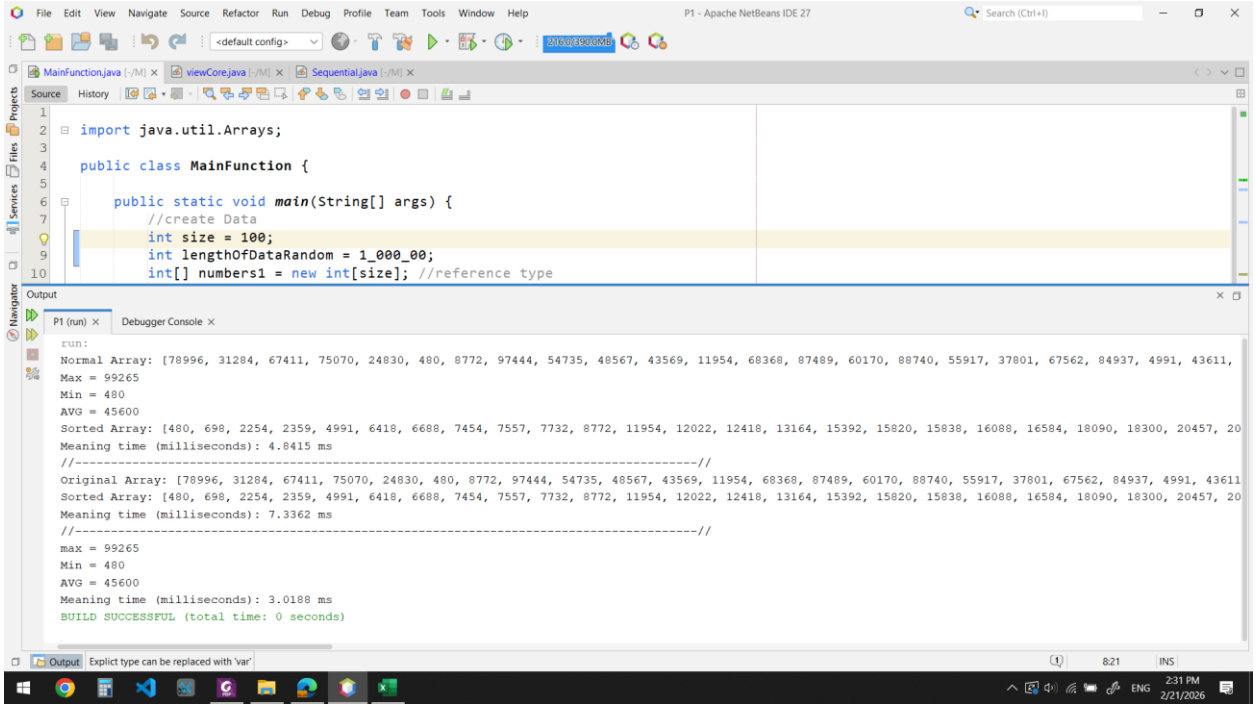
```
    public long taskAvg(int[] data) {

        long total = 0;

        long avg;

        for (int i = 0; i < data.length; i++) {

            total += data[i];

        }

        avg = total / data.length;

        return avg;

    }

    public double median(int[] data) {

        int size = data.length;

        int mid = size / 2;

        if (size % 2 != 0) {

            // For odd numbers: Pick the middle number (e.g., 5/2 = 2,
which is the 3rd index)

            return (double) data[mid];

        } else {

            // For even numbers: Add the two middle numbers together and
divide by 2

            return (double) (data[mid - 1] + data[mid]) / 2.0;

        }

    }

}
```

/* Seq */

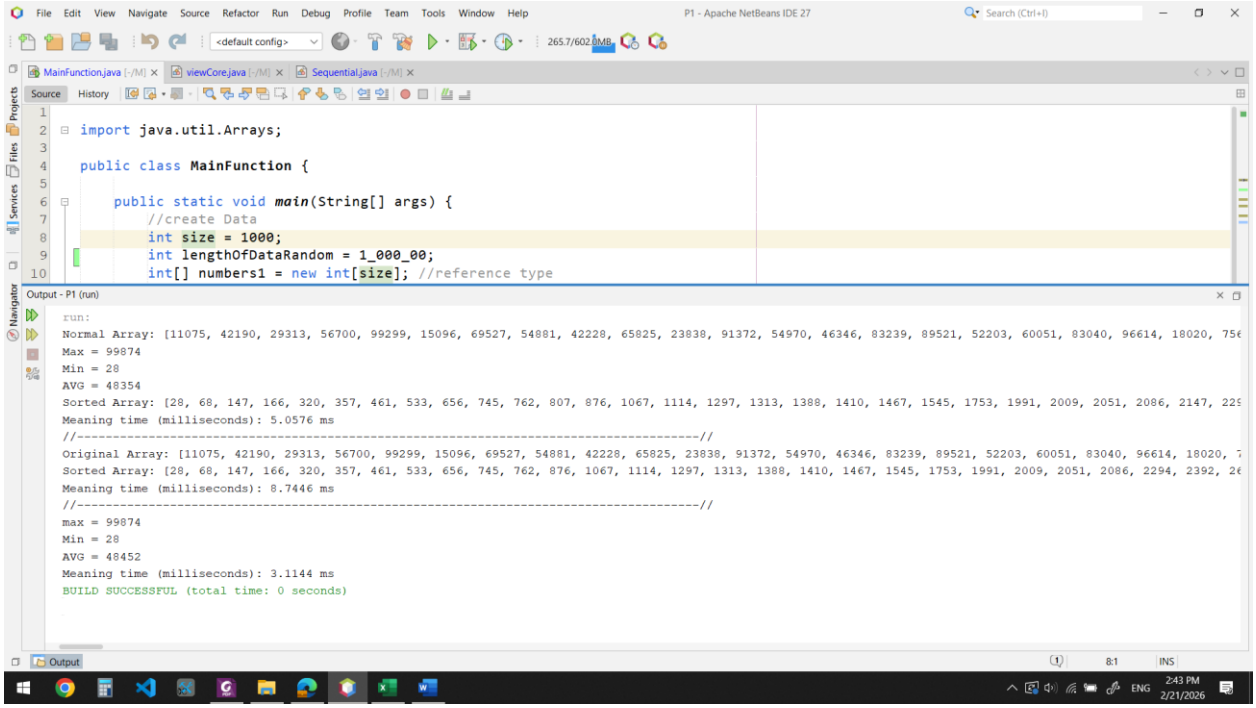
```
import java.util.Arrays;
public class Sequential {
    public void sortSequentialAndStats(int[] data) {
        long max = -1;
        long min = 9_999_999;
        long total = 0;
        long avg;
        Arrays.sort(data);
        for (int I = 0; I < data.length; i++) {
            if (data[i] > max) {
                max = data[i];
            }
        }
        for (int I = 0; I < data.length; i++) {
            if (data[i] < min) {
                min = data[i];
            }
        }
        for (int I = 0; I < data.length; i++) {
            total += data[i];
        }
        avg = total / data.length;
        System.out.println("Max = " + max);
        System.out.println("Min = " + min);
        System.out.println("AVG = " + avg);
    }//end void  }//end class Sequential
```

/* OUTPUT */

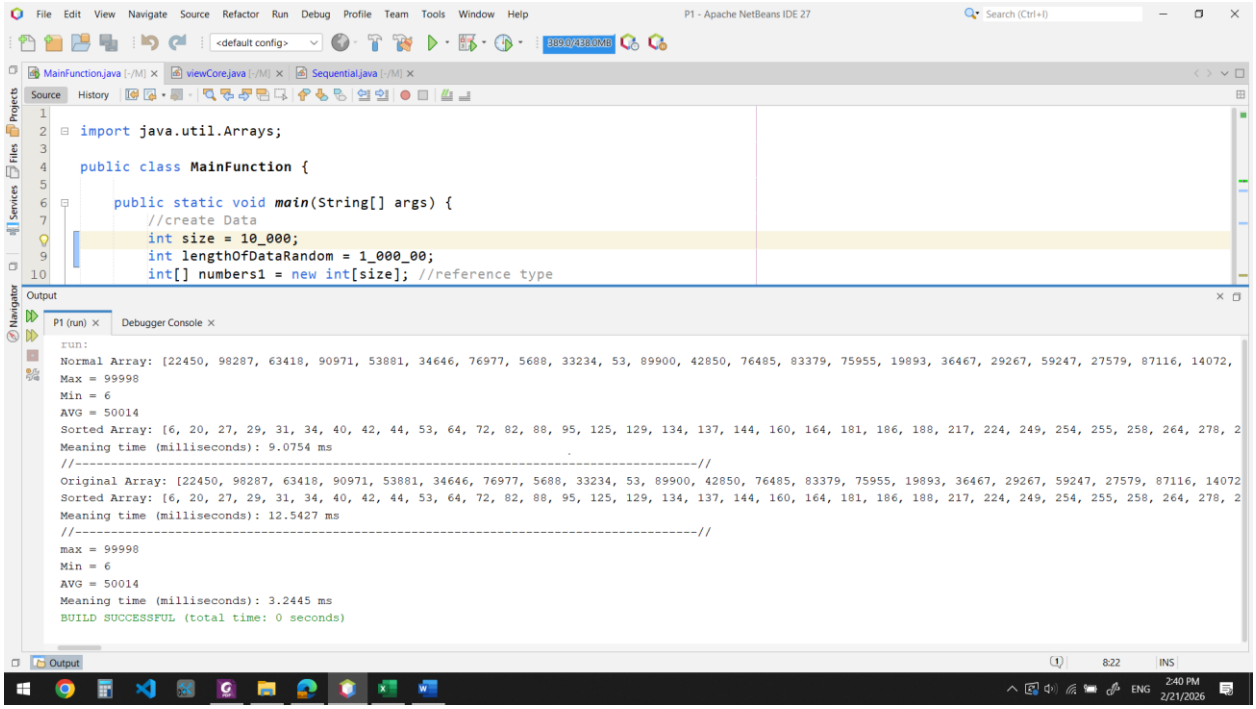
Data = 100



Data = 1,000



Data = 10,000


P1 - Apache NetBeans IDE 27
MainFunction.java
viewCore.java
Sequential.java
import java.util.Arrays;
public class MainFunction {
public static void main(String[] args) {
//create Data
int size = 10_000;
int lengthOfDataRandom = 1_000_00;
int[] numbers1 = new int[size]; //reference type
P1 (run)
Debugger Console
run:
Normal Array: [22450, 98287, 63418, 90971, 53881, 34646, 76977, 5688, 33234, 53, 89900, 42850, 76485, 83379, 75955, 19893, 36467, 29267, 59247, 27579, 87116, 14072,
Max = 99998
Min = 6
AVG = 50014
Sorted Array: [6, 20, 27, 29, 31, 34, 40, 42, 44, 53, 64, 72, 82, 88, 95, 125, 129, 134, 137, 144, 160, 164, 181, 186, 188, 217, 224, 249, 254, 255, 258, 264, 278, 2
Meaning time (milliseconds): 9.0754 ms
//----------//
Original Array: [22450, 98287, 63418, 90971, 53881, 34646, 76977, 5688, 33234, 53, 89900, 42850, 76485, 83379, 75955, 19893, 36467, 29267, 59247, 27579, 87116, 14072
Sorted Array: [6, 20, 27, 29, 31, 34, 40, 42, 44, 53, 64, 72, 82, 88, 95, 125, 129, 134, 137, 144, 160, 164, 181, 186, 188, 217, 224, 249, 254, 255, 258, 264, 278, 2
Meaning time (milliseconds): 12.5427 ms
//----------//
max = 99998
Min = 6
AVG = 50014
Meaning time (milliseconds): 3.2445 ms
BUILD SUCCESSFUL (total time: 0 seconds)

ใบงานการทดลอง : Sequential vs. Parallel Performance

วัตถุประสงค์

1. เพื่อเปรียบเทียบความแตกต่างของเวลาในการประมวลผลระหว่างการทำงานแบบลำดับ (Sequential) และแบบขนาน (Parallel)
2. เพื่อวิเคราะห์จุดคุ้มทุน (Overhead) ของการสร้างเธรดเมื่อเทียบกับขนาดของข้อมูล

ส่วนที่ 1: การประมวลผลแบบเดิม (Sequential Approach)

ให้นักศึกษาเขียนฟังก์ชันการทำงานแบบ Single Thread (ทำงานทีละอย่างตามลำดับ):

1. Sequential Sort: ใช้ Arrays.sort() บนข้อมูลทั้งหมดในเธรดเดียว
2. Sequential Stats: คำนวณ Min, Max, Mean, Average ทีละอย่างตามลำดับ (ใช้ Loop วนหาค่าทีละขั้นตอน)

ส่วนที่ 2: การประมวลผลแบบขนาน (Parallel Approach)

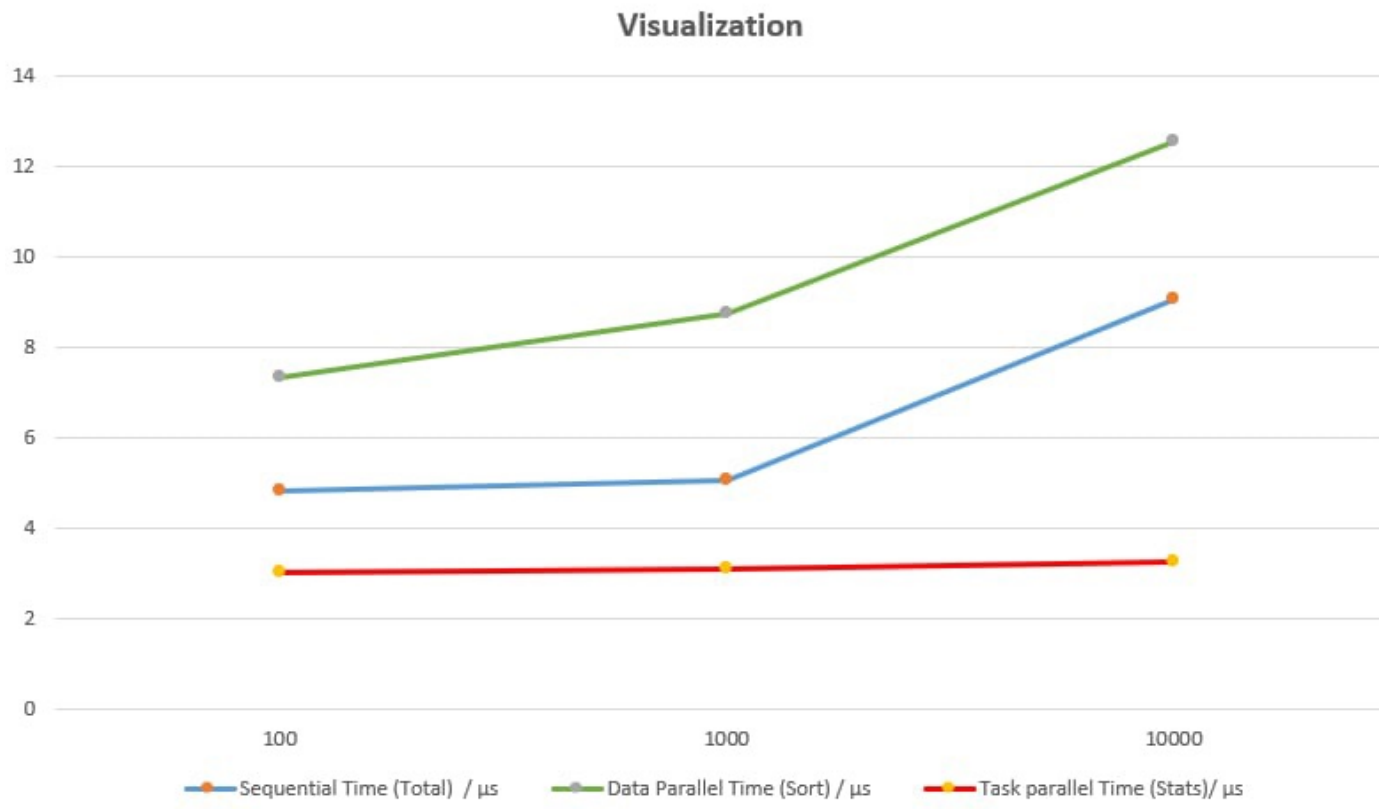
1. Data Parallelism (Sort): แบ่งกลุ่มข้อมูลตามจำนวน CPU Core และสั่ง Sort แยกเธรด
2. Task Parallelism (Stats): สั่งรันงานคำนวณสถิติทั้ง 4 อย่างแยกเธรดพร้อมกัน

ส่วนที่ 3: การบันทึกผลและเปรียบเทียบ (Data Collection)

ให้บันทึกเวลาที่ใช้ในหน่วย Microseconds (μs) เพื่อความละเอียด:

จำนวนข้อมูล (N)	Sequential Time (Total)	Data Parallel Time (Sort)	Task Parallel Time (Stats)
100	4.842	7.336	3.019
1,000	5.058	8.745	3.114
10,000	9.075	12.543	3.245

ส่วนที่ 4: การนำเสนอผลด้วยกราฟ (Visualization)

ให้นักศึกษานำข้อมูลจากตารางมาสร้างกราฟเส้น (Line Graph) โดย:

- แกน X: จำนวนข้อมูล (100, 1000, 10000)
- แกน Y: เวลาที่ใช้ในการประมวลผล
- เส้นกราฟ: เปรียบเทียบ 3 เส้น (Sequential, Data Parallel, Task Parallel)